

Recursive Jump Operators and Optimal Proof Systems

Fabian Egidy 

University of Würzburg, Germany

Abstract

We study the relationship between the existence of optimal proof systems and recursive jump operators, two central open problems in proof complexity. For a set L , an optimal proof system is a strongest proof system in terms of proof length, whereas a recursive jump operator uniformly transforms any proof system for L into a stronger one with respect to proof length, thereby witnessing non-optimality. It is clear that the existence of a recursive jump operator for L rules out optimal proof systems for L . Khaniki (FOCS 2024) is interested in the converse of this implication and explicitly poses the following question, where TAUT denotes the set of propositional tautologies.

Q: Does the non-existence of optimal proof systems for TAUT imply the existence of recursive jump operators for TAUT?

We generalize and address this question from both a relativized and an unrelativized perspective. We show that proving a positive answer for Q is provably hard by constructing the following oracle.

O: The polynomial-time hierarchy is infinite, TAUT has no optimal proof systems, and TAUT has no recursive jump operators.

This shows that Khaniki's question can not be answered in the positive by relativizable means, even under the standard complexity-theoretic assumption that the polynomial-time hierarchy is infinite.

In contrast, we obtain positive results when the question Q is posed for sets different from TAUT. We prove that the existence of recursive jump operators is upward closed under $\leq_m^P$ -reducibility, a result that so far was only known for the non-existence of optimal proof systems. Furthermore, we show that the sets known to have no optimal proof systems by Messner (STACS 1999) in fact admit recursive jump operators. Thus, essentially all sets currently known to have no optimal proof systems have recursive jump operators.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Oracles and decision trees; Theory of computation $\rightarrow$ Proof complexity; Theory of computation $\rightarrow$ Complexity classes

Keywords and phrases Relativization, Oracles, Proof Complexity, Optimal Proof Systems, Jump Operators

Digital Object Identifier 10.4230/LIPIcs.ICALP.2026.88

Category Track A: Algorithms, Complexity and Games

Related Version *Full Version:* <http://arxiv.org/abs/2606.01242>

Funding *Fabian Egidy:* supported by the German Academic Scholarship Foundation.

Acknowledgements We wish to thank Christian Glaßer for his permanent advice and helpful feedback.

1 Introduction

This paper studies the relationship of two longstanding open problems concerning Cook-Reckhow [14] proof systems. A Cook-Reckhow proof system for a set L is a polynomial time computable function f whose range is L . Two such proof systems for the same set are compared with each other using the notion of simulation. A proof system f simulates a proof system g if there exists a polynomially-bounded total function π such that $f(\pi(x)) = g(x)$



© Fabian Egidy;

licensed under Creative Commons License CC-BY 4.0

53rd International Colloquium on Automata, Languages, and Programming (ICALP 2026).

Editors: Sayan Bhattacharya, Danupon Nanongkai, Michael Benedikt, and Gabriele Puppis;

Article No. 88; pp. 88:1–88:19



Leibniz International Proceedings in Informatics

Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany



for all x . Intuitively, the proofs of f are at most polynomially longer than the proofs of g . By Krajíček and Pudlák [36], a proof system f for a set L is optimal if f simulates all proof systems for L . In other words, an optimal proof system for L has at most polynomially longer proofs than any other proof system for L .

One of the main open questions in proof complexity is whether TAUT has optimal proof systems, where TAUT denotes the set of propositional tautologies (cf. [36, 35]). In fact, it is even open whether there exists any set outside NP that has an optimal proof system (cf. [46, 31, 40]). While it is unclear which answer to this general question is to be expected [26, 9, 35], with respect to TAUT a negative answer is conjectured [36].

C1: TAUT *has no optimal proof system.*

Another concept in proof complexity are recursive jump operators. A jump operator for a set L is a function such that for any proof system f for L , $J(f)$ is a proof system for L that f cannot simulate. If additionally, J is recursive, we call J a recursive jump operator for L ¹. Intuitively, a recursive jump operator computes witnesses of non-optimality for given proof systems.

The first candidate jump operators for TAUT were proposed by Krajíček and Pudlák [36] and implicitly already by Buss [10]. Recently, Khaniki [28] proposed a new candidate jump operator based on interactive proofs. Additionally, Khaniki is interested in the relationship between jump operators and optimal proof systems. Let us present a conjecture which was already implicitly contained in the work of Krajíček and Pudlák [36] and explicitly formulated by Khaniki² [28].

C2: TAUT *has a recursive jump operator.*

It is immediate that the existence of an optimal proof system for TAUT is equivalent to TAUT possessing no non-recursive jump operators [36]. Similarly, it is clear that the existence of a recursive jump operator for TAUT implies that TAUT does not have optimal proof systems. So conjecture C2 implies conjecture C1. However, it is open whether the converse implication holds. This question is explicitly posed by Khaniki [28, Problem 10] and we state it as question Q1 below. We also generalize question Q1 to any set $L \notin \text{NP}$ ³.

Q1: *Does conjecture C1 imply conjecture C2?*

Q2: *Does the non-existence of optimal proof systems for L imply the existence of recursive jump operators for L ?*

Positive answers to these questions would show that non-optimality of proof systems always comes with a procedure computing witnesses of non-optimality, while a negative answer would separate existential and constructive notions of non-optimality in proof complexity.

In this paper we contribute to questions Q1 and Q2. We obtain hardness results for question Q1, but partial positive answers for question Q2. Before we make our contributions precise, we give an overview over Cook-Reckhow proof systems in complexity theory as well as the conjectures C1 and C2.

¹ For improved clarity, we occasionally use the term “non-recursive jump operator” for a general jump operator.

² Khaniki formulated the conjecture using partial recursive jump operators, but simultaneously shows their existence to be equivalent to the existence of recursive jump operators.

³ For all non-empty $L \in \text{NP}$ it is clear that they have optimal proof systems and no jump operators.

1.1 Previous Work

In the past 35 years optimal proof systems have been studied extensively and have shown to be connected to a wide range of research areas, including the separation of complexity classes [14, 36, 31, 39, 40, 33, 37, 13], the existence of complete sets for promise classes [45, 46, 37, 8, 43], the existence of optimal acceptors [36, 47, 39, 40], mathematical logic [36, 32, 44, 43], descriptive complexity [11], parameterized complexity [12], learning theory [41] and more (cf. Krajíček’s book on proof complexity [35]). We focus on the results from structural complexity theory that are most relevant to our work and refer the reader to the literature above for a comprehensive overview of optimal proof systems.

(Non)-existence of optimal proof systems. It is an easy fact that all sets in NP have optimal proof systems. Messner [39] shows that there are sets without optimal proof systems. He proved that for any non-polynomial time-constructible function t there are sets in $\text{coNTIME}(t)$ that do not have optimal proof systems and that all $\leq_m^P$ -hard sets for coNE do not have optimal proof systems. Surprisingly, this already covers all that we unconditionally know about the existence of optimal proof systems. Messner [40] also shows that if $\text{NP} = \text{coNP}$, then arbitrary complex sets with optimal proof systems exist. In contrast, Egidy and Glaßer [19] construct two oracles: relative to the first all sets outside non-deterministic quasi-polynomial time do not have optimal proof systems and relative to the second the polynomial-time hierarchy is infinite and all sets in $\text{PSPACE} \setminus \text{NP}$ do not have optimal proof systems. Hence, for most sets we do not know whether they have optimal proof systems and proving their existence would require to overcome at least the relativization barrier.

Proof systems for TAUT. The most interesting and best-studied proof systems are proof systems for TAUT. Krajíček and Pudlák [36] showed that $\text{NE} = \text{coNE}$ implies the existence of optimal proof systems for TAUT, which was improved to $\text{NEE} = \text{coNEE}$ by Köbler, Messner, and Torán [37]. Razborov [45] discovered the first connection between optimal proof systems and promise classes by showing that the existence of optimal proof systems for TAUT implies the existence of $\leq_m^{\text{PP}}$ -complete sets for DisjNP, i.e., the class of disjoint NP-pairs defined by Selman [48] and Selman and Grollman [25]. Initiated by this result, many further connections between proof systems for TAUT and disjoint NP-pairs were obtained [42, 3, 23, 4, 5, 21, 22, 7, 6, 9, 24]. As already mentioned above, the relationship between proof systems and promise classes has turned out to be quite fruitful. These connections in addition to various connections of optimal proof systems to bounded arithmetic (cf. [32, 44, 35]) lead into a research program initiated by Pudlák [43]. Pudlák surveys on several conjectures concerning the existence of optimal proof systems for sets (including conjecture C1) and the existence of complete sets for promise classes (like UP, $\text{NP} \cap \text{coNP}$, TFNP) and asks for a systematic investigation of their relationships. In particular, Pudlák is interested in either proving further implications between conjectures or disproving them relative to an oracle. Since then several implications have been disproved relative to an oracle [15, 18, 16, 17, 20, 23, 27].

Optimal proof systems and recursive jump operators. One approach to rule out the existence of optimal proof systems for TAUT is to provide a computable procedure that takes an arbitrary proof system f and produces an improved system that efficiently proves tautologies for which f admits only long proofs. Khaniki [28] calls such procedures efficient jump operators. Several candidate efficient jump operators have been proposed [36, 10, 34, 28]

in the hope that they are useful to prove that even strong proof systems like extended Frege are not optimal. A similar concept are hard tautology generators, that informally compute families of tautologies that a given proof system requires super-polynomial size proofs on (cf. [28, Def. 3.2] for a precise definition).

The existence of non-recursive jump operators for TAUT is equivalent to the non-existence of optimal proof systems for TAUT which is equivalent to the existence of hard tautology generators [36, 28]. Khaniki [28] extends this relationship to the “efficient” setting, i.e., Khaniki shows that the existence of all of the following concepts are equivalent: partial recursive jump operators, recursive jump operators, polynomial time computable jump operators, partial recursive hard tautology generators, recursive hard tautology generators, polynomial time computable hard tautology generators. Khaniki [28] explicitly asks whether the existence of recursive jump operators is also equivalent to the non-existence of optimal proof systems for TAUT, to which our main result contributes to.

1.2 Our Contribution

We address the questions Q1 and Q2 from a relativized and an unrelativized perspective.

Contribution to Q1. Our main contribution to Q1 is the construction of the following oracle (cf. Corollary 33), which shows that proving a positive answer for Q1 is provably hard and answers an open question by Khaniki [28].

O: The polynomial-time hierarchy is infinite, TAUT has no optimal proof systems, and TAUT has no recursive jump operators

The oracle *O* answers the question posed by Khaniki [28, Problem 10] in the sense that no relativizable proof can show the implication Q1. Even more, the hardness of Q1 remains true even if one assumes that the polynomial-time hierarchy is infinite. Note that from the point of view of answering Q1 in the negative, results like ours are the best we can hope for, because actually proving Q1 in the negative implies a proof of conjecture C1 and thus $\text{NP} \neq \text{coNP}$, which currently seems completely out of reach. Hence, a separation via oracles is a standard approach to separate two such longstanding complexity theoretic conjectures from each other, e.g., as reflected by Pudlák’s [43] approach to separate conjectures in his research program.

Contribution to Q2. In contrast to Q1, we obtain partial positive results regarding Q2, namely for sets currently known to lack optimal proof systems. Messner [39] shows the existence of sets that do not have optimal proof systems. Together with the closure under $\leq_m^P$ -reducibility for the class of all sets with optimal proof systems proved by Köbler and Messner [31], Messner obtains that all $\leq_m^P$ -hard sets for coNE do not have optimal proof systems.

First, we show that the existence of recursive jump operators is upward closed under $\leq_m^P$ -reducibility, i.e., the existence of recursive jump operators for L implies the existence of recursive jump operators for any set L' such that $L \leq_m^P L'$ (cf. Theorem 1). This is a dual result to the $\leq_m^P$ -closure result of Köbler and Messner. Second, we analyze Messner’s proof against the existence of optimal proof systems for sets. By doing so, we derive a recursive jump operator for these sets (cf. Theorem 7). In total, our results show that any set shown to not have optimal proof systems by Messner [39] in fact has recursive jump operators. This covers essentially all sets currently known to have no optimal proof systems.

1.3 Sketch of the Oracle Construction

We sketch the ideas behind the oracle construction. Our main goal is to obtain an oracle relative to which

- the polynomial-time hierarchy is infinite,
- TAUT has no optimal proof systems,
- TAUT has no recursive jump operators.

We achieve this by constructing a sparse oracle relative to which there are no $\leq_m^{\text{PP}}$ -complete disjoint NP-pairs and no recursive jump operators for TAUT. By a result of Razborov [45], our oracle then also has no optimal proof systems for TAUT. Using the approach of Egidy and Glaßer [19], we can combine our sparse oracle with an oracle of Yao [50] relative to which the polynomial-time hierarchy is infinite and obtain our desired oracle with combined properties.

We outline the construction of the sparse oracle by explaining how to achieve its two main properties separately. Combining both properties into a single oracle requires additional technical care and is achieved using a priority argument, which we defer to the full construction.

Towards no recursive jump operators for TAUT. Let $J_1, J_2, \dots$ and $P_1, P_2, \dots$ notations for the same standard enumeration of Turing transducers. For clarity, we use the enumeration J_i when referring to candidate recursive jump operators and P_i when referring to candidate proof systems. We treat all J_i as candidate recursive jump operators for TAUT and diagonalize against them. Let J_i be an arbitrary such candidate.

The key idea is to construct a witness proof system W_i that evaluates J_i on its own program code and neutralizes J_i 's jump. We achieve this by using an effective version of Kleene's [29, 30, 49] fixed-point theorem. Informally, the Turing transducer W_i works as follows:

- W_i computes $J_i(W_i) = b$, which is possible by an invocation of the fixed-point theorem.
- Depending on carefully encoded information in the oracle, W_i may either:
 - simulate the allegedly stronger proof system P_b , or
 - fall back to a fixed, baseline proof system for TAUT.

There are three ways in which J_i can fail to be a recursive jump operator. Namely, if there is some $a \in \mathbb{N}^+$ such that

F1 $J_i(a)$ does not halt.

F2 $J_i(a) = b$ and P_a is a proof system for TAUT and P_b is no proof system for TAUT.

F3 $J_i(a) = b$ and P_a, P_b are proof systems for TAUT, but P_a simulates P_b .

Note that properties such as “ P_b is a proof system for TAUT” are generally not robust under extensions of the oracle⁴. We have to account for that in the oracle construction, but for simplicity, we will ignore it in this sketch.

When diagonalizing against J_i , the oracle construction first attempts to realize Case F1 or F2 by an appropriate partial extension of the oracle. If this is not possible, we enforce Case F3 with $a := W_i$. Since Case F1 failed, $J_i(W_i) = b$ is defined. Since Case F2 failed, P_b is a proof system for TAUT. We then add some suitable code word to the oracle that allows W_i to simulate P_b . Then $J_i(W_i) = b$, W_i and P_b are proof systems for TAUT and W_i simulates P_b .

⁴ Meaning, just because P_b is a proof system for TAUT relative to some partial oracle w does not mean that this holds relative to extensions of w .

Towards no complete disjoint sets. We diagonalize against all disjoint NP-pairs for being $\leq_m^{\text{PP}}$ -complete. Given a pair of NP-machines (N_0, N_1) , we define a corresponding witness disjoint pair (A, B) whose elements are determined by the oracle on carefully chosen designated lengths. In particular, $0^n \in A$ if n is of designated length and the oracle contains words from $00\Sigma^{n-2}$. Analogous for B and $01\Sigma^{n-2}$.

For each candidate reduction function $f \in \text{FP}$, we ensure that $(A, B) \not\leq_m^{\text{PP}} (L(N_0), L(N_1))$ via f . We immediately diagonalize successfully, if we can add some word from $00\Sigma^{n-2}$ (resp., $01\Sigma^{n-2}$) to the oracle and $N_0(f(0^n))$ (resp., $N_1(f(0^n))$) rejects. If neither is possible, then both machines must accept on an exponential number of distinct oracle extensions obtained by words of length n . However, each leftmost accepting path queries only polynomially many oracle words. This allows us to extend the oracle by two words such that neither leftmost accepting path recognizes the second word. Consequently, $L(N_0)$ and $L(N_1)$ are not disjoint, and hence do not form a $\leq_m^{\text{PP}}$ -complete disjoint NP-pair.

2 Basic Definitions and Notations

Sets. Let $\Sigma := \{0, 1\}$ be the default alphabet and Σ^* be the set of finite words over Σ . The set of all (positive) natural numbers is denoted by $\mathbb{N}$ ($\mathbb{N}^+$). For $a, b \in \mathbb{N}$, we define $a\mathbb{N} + b := \{a \cdot n + b \mid n \in \mathbb{N}\}$. We write the empty set as $\emptyset$. The cardinality of a set A is denoted by $\#A$. For a set $A \subseteq \Sigma^*$ and a number $n \in \mathbb{N}$, we define $A^{\leq n} := \{w \in A \mid |w| \leq n\}$ and analogous for $=$. For a clearer notation we use $\Sigma^{\leq n}$ for $\Sigma^{*\leq n}$ and Σ^n for $\Sigma^{*=n}$. The operators $\cup$, $\cap$, and $\setminus$ denote the union, intersection and set-difference. We denote the complement of a set $A \subseteq \Sigma^*$ by $\bar{A} = \{x \in \Sigma^* \mid x \notin A\}$. We may mix the definition of sets with regular expressions, e.g., the expression $0\Sigma^*$ denotes the set $\{0x \mid x \in \Sigma^*\}$ and the expression 010^* denotes the set $\{010^n \mid n \in \mathbb{N}\}$.

Machines. We use the default model of a Turing machine in both the deterministic and non-deterministic variant. The language decided by a Turing machine M is denoted by $L(M)$. We use Turing transducer to compute functions. For a Turing transducer F we write $F(x) = y$ when on input x the transducer outputs y . Hence, a Turing transducer F computes a function and we may denote "the function computed by F " by F itself. For a Turing machine or Turing transducer M , we denote the number of steps the longest path of the computation $M(x)$ takes by $\text{time}(M(x))$. When calling a non-deterministic Turing machine M a non-deterministic Turing acceptor, then $\text{time}(M(x))$ denotes the number of steps the shortest accepting path of the computation $M(x)$ takes, because Turing acceptors may endlessly loop on some computation paths. We denote the program code of a machine M with respect to some machine enumeration by $\langle M \rangle$.

Functions. The domain and range of a function f are denoted by $\text{dom}(f)$ and $\text{ran}(f)$. We use the notion of time-constructible functions as described in [1]. A function $f: \mathbb{N} \rightarrow \mathbb{N}$ is called non-polynomial if for every polynomial p there is an $n \in \mathbb{N}$ such that $f(n) \geq p(n)$. The function computing the maximum element of a finite subset of $\mathbb{N}$ is denoted by $\max$. Also, let $\langle \cdot \rangle: \bigcup_{i \geq 0} \mathbb{N}^i \rightarrow (2\mathbb{N} + 1)$ be an injective polynomial-time computable and polynomial-time invertible sequence encoding function such that $|\langle u_1, \dots, u_n \rangle| = 2(|u_1| + \dots + |u_n| + n) + 1$.

Complexity classes. The definition of the basic complexity classes such as P, FP, NP, coNP, PH, PSPACE, E, coNE, NTIME, coNTIME and the $\leq_m^{\text{P}}$ -complete set TAUT for coNP can be found in [1]. A set $S \subseteq \Sigma^*$ is sparse if there is a polynomial p such that $\#S^{\leq n} \leq p(n)$ for all $n \in \mathbb{N}$. We denote the class of all sparse sets as SPARSE. A disjoint NP-pair is a pair (A, B) of disjoint sets in NP. Let DisjNP denote the class containing all disjoint NP-pairs.

We say that a set A is polynomial-time many-one reducible to B , denoted by $A \leq_m^p B$, if there exists a function $f \in \text{FP}$ such that $x \in A \Leftrightarrow f(x) \in B$ for all $x \in \Sigma^*$. We say that a disjoint NP-pair (A, B) is polynomial-time many-one reducible to (C, D) , denoted by $(A, B) \leq_m^{\text{pp}} (C, D)$, if there is a function $f \in \text{FP}$ such that $f(A) \subseteq C$ and $f(B) \subseteq D$. For any class $\mathcal{C}$ and any notion of reducibility $\leq$ we say that A is $\leq$ -hard for $\mathcal{C}$ when $B \leq A$ for all $B \in \mathcal{C}$. If additionally $A \in \mathcal{C}$, we say that A is $\leq$ -complete for $\mathcal{C}$.

Proof systems. We use proof systems for sets defined by Cook and Reckhow [14]. They define a function $f \in \text{FP}$ to be a proof system for $\text{ran}(f)$. Furthermore:

- A proof system g is (p-)simulated by a proof system f , denoted by $g \leq f$ (resp., $g \leq^p f$), if there exists a total function π (resp., $\pi \in \text{FP}$) and a polynomial p such that $|\pi(x)| \leq p(|x|)$ and $f(\pi(x)) = g(x)$ for all $x \in \Sigma^*$. In this context the function π is called simulation function. Note that $g \leq^p f$ implies $g \leq f$.
- A proof system f is (p-)optimal for $\text{ran}(f)$, if $g \leq f$ (resp., $g \leq^p f$) for all $g \in \text{FP}$ with $\text{ran}(g) = \text{ran}(f)$.

In order to define jump operators, we must extend the definition of proof systems from polynomial-time computable functions to Turing transducers. There are two sensible options. First, one may consider any Turing transducer computing an FP function as a proof system, even if the transducer itself does not run in polynomial time. Alternatively, one may only consider Turing transducers running in polynomial time as proof systems. In this paper, we adopt the second option and thus identify proof systems with polynomial-time Turing transducers.

Recursive jump operator. A jump operator for a set L is a function $J: \mathbb{N}^+ \rightarrow \mathbb{N}^+$ such that on input a , if a is the code of a proof system F for L , then $J(a)$ is the code of a proof system G for L such that F cannot simulate G .⁵ If additionally J is recursive, then J is a recursive jump operator.

3 Recursive Jump Operators for Sets Without Optimal Proof Systems

Köbler and Messner [31] show that the class of sets with optimal proof systems is downward-closed under $\leq_m^p$ -reducibility, with the trivial exception of $\emptyset$, which admits no proof system by definition. Our first result follows up on this by showing that the class of sets with recursive jump operators is upward-closed under $\leq_m^p$ -reducibility. Second, we extract recursive jump operators from Messner's [39] proof against the existence of optimal proof systems for sets in $\text{coNTIME}(t)$ for non-polynomial and time-constructible t . Combining both results shows that the sets known to not have optimal proof system by Messner [39] also have recursive jump operators.

For the rest of this section, let $P_1, P_2, \dots$ be some standard enumeration of deterministic Turing transducers.

► **Theorem 1.** *If $A \neq \emptyset$ has a recursive jump operator and $A \leq_m^p B$, then B also has a recursive jump operator.*

⁵ Note that the choice of which Turing transducers are regarded as a proof system is relevant here, since this determines the set of valid outputs as well as the set of inputs on which the jump operator is required to produce valid outputs.

Proof. Let J denote the recursive jump operator for A and let $A \leq_m^P B$ via some function $f \in \text{FP}$. Let g be an arbitrary proof system for B and let $\perp$ be an arbitrary element from A . Define

$$g'(\langle x, w \rangle) := \begin{cases} x, & \text{if } g(w) = f(x) \\ \perp, & \text{otherwise} \end{cases}.$$

Observe that g' is a proof system for A , because $x \in \text{ran}(g')$ if and only if $f(x) \in \text{ran}(g) = B$. Let $\langle g' \rangle$ denote the code of a Turing transducer computing g' . Let

$$\langle h' \rangle := J(\langle g' \rangle),$$

i.e., h' is a proof system for A such that g' does not simulate h' . Finally, define

$$h(w) := \begin{cases} f(h'(w')), & \text{if } w = 1w' \\ g(w'), & \text{if } w = 0w' \end{cases}$$

Clearly h is a proof system for B . We show that g does not simulate h .

Suppose for the sake of contradiction that g simulates h via π , i.e., $h(w) = g(\pi(w))$ for all $w \in \Sigma^*$. Then $g(\pi(1w)) = h(1w) = f(h'(w))$. Consequently, $g'(\langle h'(w), \pi(1w) \rangle) = h'(w)$, showing that any h' -proof has an at most polynomially longer g' -proof. This is a contradiction to g' not simulating h' , so g does not simulate h .

Above construction provides a recursive jump operator J' for B . Given any Turing transducer code a , J' can compute the code b of a Turing transducer working like g' where P_a is used for g . By assumption, J' can use J to compute $c := J(b)$. Finally, J' can compute the code d of a Turing transducer computing h where P_a is used for g and P_c is used for h' . Hence, J' is recursive. Also, observe that if P_a is a proof system for B , then J' can compute the codes b , c , and d such that P_b , P_c , and P_d are polynomial-time computable. Hence, J' is recursive and if P_a is a proof system for B , then $J'(a) = d$ where P_d is a proof system for B that is not simulated by P_a . ◀

Above theorem can be helpful when approaching question Q1.

► **Corollary 2.** *Let $L \neq \emptyset$ and $L \leq_m^P \text{TAUT}$. If L has recursive jump operators, then TAUT has recursive jump operators.*

Next, we present the two most prominent results showing that there are sets without optimal proof systems.

► **Theorem 3** ([39, 40]). *Let $t: \mathbb{N} \rightarrow \mathbb{N}$ be non-polynomial and time-constructible. Then there is a set $L \in \text{coNTIME}(t)$ for which no optimal proof system exists.*

► **Corollary 4** ([39, 40]). *No set $\leq_m^P$ -hard for coNE has an optimal proof system.*

We prove that the sets from Theorem 3 and Corollary 4 without optimal proof systems admit recursive jump operators. For this, we first define the sets considered in the proof of Theorem 3 and follow with our result.

► **Definition 5.** *Let $t: \mathbb{N} \rightarrow \mathbb{N}$ be non-polynomial and time-constructible. Let $N_1, N_2, \dots$ be a standard enumeration of non-deterministic Turing acceptors, and let U be a universal non-deterministic acceptor that on input $0^i 1x$ simulates $N_i(0^i 1x)$ such that $\text{time}(U(0^i 1x)) \leq c_i \text{time}(N_i(0^i 1x)) + c_i$ for some constant c_i . For any $i \in \mathbb{N}^+$ define*

$$L_{i,t} := \{x \in 0^i 10^* \mid U(x) \text{ does not accept in less than } t(|x|) \text{ steps}\}$$

Define $L_t := \bigcup_{i \in \mathbb{N}^+} L_{i,t}$.

Note that by definition a time-constructible function t must satisfy $t(n) \geq n$.

► **Observation 6.** *It holds that $L_t \in \text{coNTIME}(t)$.*

► **Theorem 7.** *Let $t: \mathbb{N} \rightarrow \mathbb{N}$ be non-polynomial and time-constructible. Then L_t has a recursive jump operator.*

Proof. For $i \in \mathbb{N}^+$ let $D[i]$ denote a polynomial time Turing transducer with range $0^i 10^*$ and $D[i](0^i 10^j) = 0^i 10^j$ for all $j \in \mathbb{N}$. For $a \in \mathbb{N}^+$ let $N[a]$ denote a non-deterministic Turing acceptor that on input y computes $P_a(x) = y'$ for all x and accepts on paths where $y = y'$. Then $N[a]$ accepts L_t if and only if $\text{ran}(P_a) = L_t$. It is clear that given $a \in \mathbb{N}^+$ one can compute $i \in \mathbb{N}^+$ such that $N_i = N[a]$.

Algorithm 1 defines how the jump operator J for L_t operates.

■ **Algorithm 1** Jump operator J for L_t .

-
- 1: **Input:** $a \in \mathbb{N}^+$
 - 2: Compute code i such that $N_i = N[a]$
 - 3: Compute code b such that $P_b(x) = \begin{cases} P_a(x') & \text{if } x = 1x' \\ D[i](x') & \text{if } x = 0x' \end{cases}$
 - 4: **return** b
-

It is clear that J is recursive and that b can be computed such that P_b 's runtime is in the order of magnitude of P_a and $D[i]$. We show that J is also a jump operator for L_t . Let $a \in \mathbb{N}^+$ such that P_a is a proof system for L_t . Consequently, $N[a] = N_i$ accepts L_t . Then it holds that

$$0^i 10^* \subseteq L_t, \quad (1)$$

because if $0^i 10^j \notin L_t$ for some $j \in \mathbb{N}$, then $U(0^i 10^j)$ accepts meaning that also $N_i(0^i 10^j)$ accepts, a contradiction to N_i accepting L_t . Furthermore,

$$c_i \text{time}(N_i(x)) + c_i \geq \text{time}(U(x)) \geq t(|x|) \text{ for } x \in 0^i 10^*, \quad (2)$$

because if $U(x)$ accepts in less than $t(|x|)$ steps, then $x \notin L_t$, contradicting $0^i 10^* \subseteq L_t$.

By (1), by $\text{ran}(D[i]) = 0^i 10^*$, and by P_a being a proof system for L_t , it follows that P_b is also a proof system for L_t . Furthermore, for $x \in 0^i 10^*$, $P_b(0x) = D[i](x) = x$, so any $x \in 0^i 10^*$ has a P_b -proof of length $|x| + 1$. We show that the proof system P_a can not simulate the proof system P_b , because of P_b 's short proofs for $0^i 10^*$.

Assume for the sake of contradiction that P_a simulates P_b via the function π whose output-length is bounded by the polynomial q . Let p be the polynomial bounding the runtime of P_a . For any $x \in L_t$, let $\hat{x}$ be the lexicographically shortest P_a -proof for x . Since $N_i(x)$ takes $|\hat{x}|$ steps to guess the shortest P_a -proof for x and at most $p(|\hat{x}|)$ steps to compute $P_a(\hat{x})$, from (2) we get

$$c_i p(|\hat{x}|) + |\hat{x}| + c_i \geq c_i \text{time}(N_i(x)) + c_i \geq t(|x|) \text{ for } x \in 0^i 10^*.$$

Since P_a simulates P_b via π , for all $x \in 0^i 10^*$ it also holds that

$$|\hat{x}| \leq q(|x| + 1).$$

Let $d_i := \max\{t(n) \mid n \leq i\}$. Then for all $n \in \mathbb{N}$ it holds that

$$c_i p(q(n+1)) + q(n+1) + c_i + d_i \geq t(n).$$

Note that $c_i p(q(n+1)) + q(n+1) + c_i + d_i$ is polynomial in n . This contradicts that t is non-polynomial. So the assumption is false and P_a does not simulate P_b .

In total, J is recursive and when a is code of a proof system P_a for L_t , then $b = J(a)$ is a code of a proof system P_b for L_t and P_a does not simulate P_b . Hence, J is a recursive jump operator for L_t . $\blacktriangleleft$

► **Corollary 8.** *All sets $\leq_m^P$ -hard for coNE have recursive jump operators.*

Proof. Let $t(n) := 2^n$. By Theorem 7 and Observation 6, there is $L_t \in \text{coNTIME}(t) \subseteq \text{coNE}$ that has a recursive jump operator. Let L be an arbitrary $\leq_m^P$ -hard set for coNE. Then $L_t \leq_m^P L$ and thus Theorem 1 gives that L also has recursive jump operators. $\blacktriangleleft$

4 Oracle Against a Positive Answer to Q1

In this section, we construct a sparse oracle relative to which DisjNP has no $\leq_m^{\text{PP}}$ -complete sets and TAUT has no recursive jump operator. By a result of Razborov [45], there are also no optimal proof systems for TAUT relative to the sparse oracle. Using the approach of Egidy and Glaßer [19], we can combine this sparse oracle with an oracle relative to which the polynomial-time hierarchy is infinite and obtain an oracle that has the combined properties. First, we introduce some further definitions and oracle-specific notations and follow with the rigorous construction of the oracle.

4.1 Notation for the Oracle Construction

We relativize the concept of Turing machines and Turing transducers by giving them access to a write-only oracle tape. We say that two oracle transducers F and G are equivalent, denoted by $F \equiv G$, if they compute the same function. We relativize machines, complexity classes, proof systems, jump operators, and (p-)simulation by defining them over machines with oracle access, i.e., whenever a Turing machine or Turing transducer is part of a definition, we replace them by an oracle Turing machine or an oracle Turing transducer. We indicate the access to some oracle O in the superscript of the mentioned concepts, i.e., C^O for a complexity class C and M^O for a Turing machine or Turing transducer M . We sometimes omit the oracles in the superscripts, e.g., when sketching ideas in order to convey intuition, but never in actual proofs. We also transfer all notations to the respective oracle concepts. Note that the coNP-complete set TAUT can be generalized such that for every oracle O , TAUT^O is coNP O -complete. Fix some strictly monotone increasing polynomial p_t such that TAUT^O is decidable in non-deterministic time p_t relative to all oracles O .

Words and sets. We denote the length of a word $w \in \Sigma^*$ by $|w|$. The empty word has length 0 and is denoted by ε . The $(i+1)$ -th letter of a word w for $0 \leq i < |w|$ is denoted by $w(i)$, i.e., $w = w(0)w(1) \cdots w(|w|-1)$. If v is a (strict) prefix of w , we write $v \sqsubseteq w$ ($v \sqsubset w$) or $w \sqsupseteq v$ ($w \sqsupset v$).

We identify Σ^* with $\mathbb{N}$ through a polynomial-time computable and polynomial-time invertible bijection $\text{enc}: \Sigma^* \rightarrow \mathbb{N}$ defined as $\text{enc}(w) = \sum_{i < |w|} (1 + w(i))2^i$. Thus, we can treat words from Σ^* as numbers from $\mathbb{N}$ and vice versa, which allows us to use notations, relations and operations of words for numbers and vice versa (e.g., we can define the length

of a number by this). In particular, $\text{ran}(\langle \cdot \rangle) \subseteq 0\Sigma^*$. Expressions like 0^i and 1^i are ambiguous, because they can be interpreted as words from Σ^* or numbers. We resolve this ambiguity by the context or explicitly naming the interpretation. A word $w \in \Sigma^*$ can be interpreted as a set $\{i \in \mathbb{N} \mid w(i) = 1\}$ and subsequently as a partial oracle, which is defined for all words up to $|w| - 1$. So $|w|$ is the first word that w is not defined for, i.e., $|w| \in w1$ and $|w| \notin w0$. During the oracle construction we often use words from Σ^* to denote partially defined oracles. In particular, oracle queries for undefined words of a partial oracle are answered negatively.

For a deterministic Turing machine or transducer F and an oracle O , we define $Q(F^O(x))$ as the set of words queried to the oracle by the computation $F^O(x)$. For a non-deterministic Turing machine N , we define $Q(N^O(x))$ as the set of words queried on the leftmost accepting computation path of $N^O(x)$. If $N^O(x)$ rejects, then $Q(N^O(x))$ is the empty set. Note that in the context of Q we view a chained computation like $N^O(F^O(x))$ as one computation that first computes $F^O(x) = y$ and then $N^O(y)$.

Functions. A function f' is an extension of a function f , denoted by $f \sqsubseteq f'$ or $f' \sqsupseteq f$, if $\text{dom}(f) \subseteq \text{dom}(f')$ and $f(x) = f'(x)$ for all $x \in \text{dom}(f)$. If $x \notin \text{dom}(f)$, then $f \cup \{x \mapsto y\}$ denotes the extension f' of f such that $f'(z) = f(z)$ for $z \neq x$ and $f'(x) = y$. We define polynomial functions $p_i: \mathbb{N} \rightarrow \mathbb{N}$ for $i \in \mathbb{N}^+$ by $p_i(n) := n^i + i$.

Definite computations. A computation $F^w(x)$ is called definite, if w is defined for all words of length less than or equal to the runtime bound of $F^w(x)$. A computation definitely accepts (resp., rejects), if it is definite and accepts (resp., rejects). We may combine computations to more complex expressions and call those definite, if every individual computation is definite, e.g., “ $N^w(F^w(x))$ is definite” means $F^w(x)$ outputs some y and is definite, and $N^w(y)$ is definite.

Enumerations. For any oracle O let $\{F_i^O\}_{i \in \mathbb{N}^+}$ be a standard enumeration of polynomial time oracle Turing transducers, where F_i^O has running time exactly p_i . For any oracle O let $\{N_i^O\}_{i \in \mathbb{N}^+}$ be a standard enumeration of non-deterministic polynomial time oracle Turing machines, where N_i^O has running time exactly p_i . For any oracle O , let $\{J_i^O\}_{i \in \mathbb{N}^+}$ be a standard enumeration of oracle Turing transducers and let $\{P_i^O\}_{i \in \mathbb{N}^+}$ be a different notation for the same enumeration. For readability, we will use the machines J_i when referring to candidate jump operators and the machines P_i when referring to candidate proof systems, although J_i is the same machine as P_i .

4.2 Oracle Construction

For this section, let $A \subseteq 1\Sigma^*$ (equivalently $A \subseteq 2\mathbb{N}$) be arbitrary. We will treat A as some base oracle already present in the beginning of the construction. We will construct a sparse oracle $O \subseteq 0\Sigma^*$ (equivalently $O \subseteq 2\mathbb{N} + 1$), such that the desired properties hold relative to $O \cup A$.

Witness pairs. For each candidate disjoint NP-pair (C, D) we define a witness pair $(A_m, B_m) \in \text{DisjNP}$ showing that $(A_m, B_m) \not\leq_m^{\text{PP}} (C, D)$, i.e., (A_m, B_m) is a witness that (C, D) is no $\leq_m^{\text{PP}}$ -complete pair for DisjNP.

► **Definition 9 (Stages).** Define $e: \mathbb{N} \rightarrow \mathbb{N}$ such that $e(0) := 2$ and $e(n) := 2^{e(n-1)}$ for $n \in \mathbb{N}^+$. For $m \in \mathbb{N}$ define $H_m := \bigcup_{i \in \mathbb{N}} e(\langle m, i \rangle)$. Define $\mathcal{H} = \bigcup_{m \in \mathbb{N}} H_m$.

► **Observation 10 (Properties of H_m).** $\mathcal{H} \in \text{P}$ and $H_m \in \text{P}$ for all $m \in \mathbb{N}$.

► **Definition 11** (Witness pair). For an oracle O and $m \in \mathbb{N}$ define

$$A_m^O := \{0^n \mid n \in H_m \text{ and } O^{=n} \cap 00\Sigma^{n-2} \neq \emptyset\}$$

$$B_m^O := \{0^n \mid n \in H_m \text{ and } O^{=n} \cap 01\Sigma^{n-2} \neq \emptyset\}$$

► **Observation 12.** For all $m \in \mathbb{N}$ and all oracles O , it holds that $A_m^O, B_m^O \in \text{NP}^O$.

► **Observation 13.** Let $m \in \mathbb{N}$ and O be an oracle. If for all $n \in H_m$ it holds that $O^{=n} \cap 00\Sigma^{n-2} = \emptyset$ or $O^{=n} \cap 01\Sigma^{n-2} = \emptyset$, then $(A_m^O, B_m^O) \in \text{DisjNP}^O$.

Witness proof systems. For each candidate recursive jump operator J_i we define many witness proof systems $W_{i,m}$ possibly witnessing that J_i is no recursive jump operator for TAUT, i.e., $J_i(\langle W_{i,m} \rangle) = b$ and either P_b is no proof system for TAUT or $W_{i,m}$ simulates P_b .

► **Theorem 14** (Relativized effective version of Kleene's fixed-point theorem [29, 30, 49]). There is a recursive function $\text{fp}: \mathbb{N}^+ \rightarrow \mathbb{N}^+$ such that for all $i \in \mathbb{N}^+$ it holds that if J_i is total, then $P_{\text{fp}(i)}^O \equiv P_{J_i(\text{fp}(i))}^O$ relative to any oracle O .

We use this fixed-point theorem to define our witness proof systems $W_{i,m}$ such that they evaluate J_i on their own program code. For this, let $i, m \in \mathbb{N}^+$ and let $s_{i,m}: \mathbb{N}^+ \rightarrow \mathbb{N}^+$ be a function that returns the code of the oracle-program defined in Algorithm 2 when given $z \in \mathbb{N}^+$ as input. Note that O is a placeholder for an arbitrary oracle, let F be a proof system for TAUT relative to all oracles, and recall that p_t is the polynomial bounding the number of steps to non-deterministically decide TAUT relative to any oracle. We provide some intuition of Algorithm 2 after Observation 17.

■ **Algorithm 2** Witness proof system with code $s_{i,m}(z)$.

```

1: Input:  $\langle x, y \rangle \in \Sigma^*$ 
2: Compute  $b := J_i^O(z)$                                 ▷ obtain “jumped” proof system
3: if  $\langle 0^i, 0^m, 0^n \rangle \in O$  for any  $n < |y|$  then        ▷ oracle permits simulation
4:   if  $\langle 1^i, 1^m, 1^n \rangle \notin O$  for all  $n \leq p_t(|y|)$  then  ▷ permission not revoked
5:     if  $P_b^O(x)$  halts after  $\leq |y|$  steps then           ▷ ensures polytime simulation
6:       return  $P_b^O(x)$                                      ▷ simulate “jumped” proof system
7:     else
8:       return  $F^O(x)$ 
9:     end if
10:  end if
11: end if
12: return  $F^O(x)$ 

```

For all $i, m \in \mathbb{N}^+$, let $\langle s_{i,m} \rangle$ denote the program code of a polynomial-time Turing transducer computing $s_{i,m}$. The following observation shows that such a Turing transducer exists.

► **Observation 15.** Let O be an arbitrary oracle. For all $i, m \in \mathbb{N}^+$ it holds that

- (i) $s_{i,m} \in \text{FP}$.
- (ii) $P_{\text{fp}(\langle s_{i,m} \rangle)}^O \equiv P_{s_{i,m}(\text{fp}(\langle s_{i,m} \rangle))}^O$.

Proof. Let $i, m \in \mathbb{N}^+$ be arbitrary. It is clear that (i) holds. Property (ii) follows by the totality of $s_{i,m}$ and Theorem 14 invoked with $\langle s_{i,m} \rangle$ for i . ◀

► **Definition 16** (Witness proof systems). Let O be an arbitrary oracle and let $i, m \in \mathbb{N}^+$. Define $W_{i,m}^O := P_{\text{fp}(\langle s_{i,m} \rangle)}^O \equiv P_{s_{i,m}(\text{fp}(\langle s_{i,m} \rangle))}^O$.

► **Observation 17.** The witness proof system $W_{i,m}$ has code $\text{fp}(\langle s_{i,m} \rangle)$ and works like Algorithm 2 for i, m , and $z := \text{fp}(\langle s_{i,m} \rangle)$.

With this, for every candidate recursive jump operator J_i , we have infinitely many witness proof systems $W_{i,m}$ behaving like Algorithm 2 where $z = \langle W_{i,m} \rangle$. So $W_{i,m}$ first evaluates J_i on its own program code and obtains the code of a supposedly stronger proof system P_b . Depending on information in the oracle, $W_{i,m}$ is allowed to simulate P_b . Here, the oracle should only allow the simulation of P_b if $\text{ran}(P_b) \subseteq \text{TAUT}$. Lastly, $W_{i,m}$ only simulates P_b if the simulation can be done in a polynomial number of steps, ensuring that $W_{i,m}$ remains polynomial-time computable.

The following results show that if P_b is a proof system and the oracle has the right conditions, $W_{i,m}$ is a proof system for TAUT that simulates P_b .

► **Lemma 18.** For all $i, m \in \mathbb{N}^+$ and all oracles O it holds that if $b := J_i^O(\text{fp}(\langle s_{i,m} \rangle))$ is defined, then $W_{i,m}^O$ is total and runs in polynomial time. If additionally $\text{ran}(P_b^O) \subseteq \text{TAUT}^O$, then $W_{i,m}^O$ is a proof system for TAUT^O .

Proof. Consider $W_{i,m}^O$. Line 2 does not depend on the size of the input and halts by assumption, thus requiring only constant time. Lines 3 and 4 are about at most polynomially many polysize oracle queries. Lines 5 and 6 are about simulating computations for a polynomially bounded number of steps. Finally, lines 8 and 12 simulate a proof system for TAUT^O . Hence, $W_{i,m}^O$ runs in polynomial time.

Line 2 computes the number b from the assumption. Any output is either computed by P_b^O or F^O , whose ranges are subsets of TAUT^O . Hence, $\text{ran}(W_{i,m}^O) \subseteq \text{TAUT}^O$. Furthermore, line 3 never evaluates to TRUE for inputs of the form $\langle x, \varepsilon \rangle$, so line 12 is executed for all $x \in \Sigma^*$ at least once, resulting in $\text{ran}(W_{i,m}^O) \supseteq \text{ran}(F^O) = \text{TAUT}^O$. Consequently, $\text{ran}(W_{i,m}^O) = \text{TAUT}^O$. Together with $W_{i,m}^O$ running in polynomial time, we get that $W_{i,m}^O$ is a proof system for TAUT^O . ◀

► **Lemma 19.** Let $i, m \in \mathbb{N}^+$ and O be an arbitrary oracle. Let $b := J_i^O(\text{fp}(\langle s_{i,m} \rangle))$ be defined and let P_b^O be a proof system for TAUT^O . If $\langle 0^i, 0^m, 0^n \rangle \in O$ for some $n \in \mathbb{N}$ and $\langle 1^i, 1^m, 1^n \rangle \notin O$ for all $n \in \mathbb{N}$, then it holds that $W_{i,m}^O$ is a proof system for TAUT^O simulating P_b^O .

Proof. By Lemma 18 and P_b^O being a proof system for TAUT^O , it follows that $W_{i,m}^O$ is also a proof system for TAUT^O .

Let p be a strictly monotone increasing polynomial bounding the runtime of the proof system P_b^O . Let x be arbitrary such that line 3 evaluates to TRUE on input $\langle x, 0^{p(|x|)} \rangle$. Only finitely many x do not satisfy this. Also by assumption, line 4 always evaluates to TRUE. Then for the input $\langle x, 0^{p(|x|)} \rangle$ line 5 evaluates to TRUE, because $P_b^O(x)$ halts in $\leq p(|x|)$ steps. Hence, for all but finitely many x it holds that $W_{i,m}(\langle x, 0^{p(|x|)} \rangle) = P_b^O(x)$. Consequently, $W_{i,m}^O$ has at most polynomially longer proofs than P_b^O , so $W_{i,m}^O$ simulates P_b^O . ◀

Valid oracles. During the construction of the oracle, we successively add requirements that we maintain. These are specified by a function $r: \mathbb{N}^+ \times \mathbb{N} \times \mathbb{N} \mapsto \mathbb{N}$, called *requirement function*. Recall that $A \subseteq 1\Sigma^*$ denotes an arbitrary base oracle. An oracle w is r -valid for a requirement function r , if it satisfies the following requirements (let $i, j, m \in \mathbb{N}^+$):

V1 $w \subseteq 0\Sigma^*$ and $\#(w \cap \Sigma^n) \leq 2$ for all $n \in \mathbb{N}^+$.

(Meaning: $w \subseteq 2\mathbb{N} + 1$ and w is sparse.)

V2 If $\langle 0^i, 0^m, 0^n \rangle \in w$ or $\langle 1^i, 1^m, 1^n \rangle \in w$ for some $n \in \mathbb{N}$, then $n > \text{time}(J_i^{w \cup A}(\text{fp}(\langle s_{i,m} \rangle)))$.

(Meaning: Codings for $W_{i,m}$ can not be queried during the simulation in line 2.)

V3 If $r(i, 0, 0) = m > 1$, then $\langle 1^i, 1^m, 1^n \rangle \notin w$ for all $n \in \mathbb{N}$.

(Meaning: Here, $W_{i,m}$ commits to also return by line 6.)

V4 If $r(i, j, 0) = m > 0$, then $\#(w \cap \Sigma^n) \leq 1$ for all $n \in H_m$.

(Meaning: (A_m, B_m) is disjoint relative to A combined with any extension of w .)

We will prove that V1, V2, V3 and V4 are satisfied for various oracles and requirement functions. To prevent confusion, we use the notation $V1(w)$, $V2(w)$, $V3(w, r)$ and $V4(w, r)$ to clearly state the referred oracle w and requirement function r .

► **Observation 20.** If w is an r -valid oracle, then $w0$ is r -valid.

► **Observation 21.** If w is r -valid, then w is also r' -valid for any $r' \sqsubseteq r$.

► **Observation 22.** If w is r -valid and $v \sqsubseteq w$, then v is also r -valid.

Oracle construction. The oracle construction will take care of the following set of tasks $\{\tau_{i,j,k} \mid i \in \mathbb{N}^+, j, k \in \mathbb{N}\}$ that are defined below. Let $\mathcal{T}$ be an enumeration of these tasks with the property that $\tau_{i,j,k}$ appears earlier than $\tau_{i,j,k+1}$. In each step we treat the smallest task in the order specified by $\mathcal{T}$, and after treating a task we remove it from $\mathcal{T}$.

In step $s = 0$ we define $w_0 := \varepsilon$ and r_0 as the nowhere defined requirement function. In step $s > 0$ we define $w_s \supsetneq w_{s-1}$ and $r_s \supsetneq r_{s-1}$ such that w_s is r_s -valid by treating the earliest task τ in $\mathcal{T}$ and removing it from $\mathcal{T}$. We do this according to the following procedure (let $i, j, k \in \mathbb{N}^+$):

Task $\tau_{i,0,0}$

1. If there exists an r_{s-1} -valid partial oracle $v \supsetneq w_{s-1}$ such that there are $a \in \mathbb{N}^+$ and $x \in \Sigma^*$ with
 - a. P_a is a proof system for TAUT relative to A combined with any fully defined r_{s-1} -valid extension of v ,
 - b. $J_i^{v \cup A}(a) = b$ is definite,
 - c. $P_b^{v \cup A}(x) \notin \text{TAUT}^{v \cup A}$ is definite,
 then define $w_s := v$ and $r_s := r_{s-1} \cup \{(i, 0, 0) \mapsto 0\}$.
 (Meaning: J_i will not be a jump operator, because J_i maps some code a to some code b , while P_a will be and P_b will not be a proof system for TAUT relative to the final oracle.)
2. Otherwise, if there exists an $a \in \mathbb{N}^+$ such that $J_i(a)$ never halts relative to A combined with any r_{s-1} -valid partial extension of w_{s-1} , then define $w_s := w_{s-1}0$ and $r_s := r_{s-1} \cup \{(i, 0, 0) \mapsto 1\}$.
 (Meaning: J_i will not be a recursive jump operator, because J_i will not be total.)
3. Otherwise,
 - a. let $m := |w_{s-1}| + 2$,
 - b. let $v' \supsetneq w_{s-1}$ be the smallest r_{s-1} -valid partial oracle such that $J_i^{v' \cup A}(\text{fp}(\langle s_{i,m} \rangle))$ halts in less than $\|v'\|$ steps,
 - c. and let v be an extension of v' via zeros such that $\langle 0^i, 0^m, 0^{\|v'\|} \rangle \in v1$.
 Then define $w_s := v1$ and $r_s := r_{s-1} \cup \{(i, 0, 0) \mapsto m\}$.
 (Meaning: Choose a sufficiently large m such that w_{s-1} can not contain words of the form $\langle 0^i, 0^m, \cdot \rangle$ and let w_s contain the permission such that $W_{i,m}$ can return by line 6, thereby ruling out J_i as a jump operator. Statement 3b is possible since Case 2 failed.)

Task $\tau_{i,j,0}$

1. If there exists an r_{s-1} -valid partial oracle $v \supsetneq w_{s-1}$ such that there is an x with $N_i^{v \cup A}(x)$ and $N_j^{v \cup A}(x)$ accept definitely, then define $w_s := v$ and $r_s := r_{s-1} \cup \{(i, j, 0) \mapsto 0\}$. Remove all tasks $\tau_{i,j,k}$ with $k \in \mathbb{N}^+$ from $\mathcal{T}$.
(Meaning: The pair $(L(N_i), L(N_j))$ is not disjoint.)
2. Otherwise, define $w_s := w_{s-1}0$ and $r_s := r_{s-1} \cup \{(i, j, 0) \mapsto |w_s| + 2\}$.
(Meaning: Assign the pair $(L(N_i), L(N_j))$ some witness pair $(A_{|w_s|+2}, B_{|w_s|+2})$ and promise to keep the witness-pair disjoint.)

Task $\tau_{i,j,k}$

1. Let $m := r_{s-1}(i, j, 0)$ and let $v \supseteq w_{s-1}$ be r_{s-1} -valid such that $\|v\| = 0^n$ with $n \in H_m$ and $2^{n-2} > 6(p_{i+j}(p_k(n)))^2 + 1$. Choose
 - a. either $z \in 00\Sigma^{n-2}$ such that $N_i^{v \cup \{z\} \cup A}(F_k^{v \cup \{z\} \cup A}(0^n))$ rejects
 - b. or $z \in 01\Sigma^{n-2}$ such that $N_j^{v \cup \{z\} \cup A}(F_k^{v \cup \{z\} \cup A}(0^n))$ rejects.
 Then define $r_s := r_{s-1} \cup \{(i, j, k) \mapsto 0\}$ and define $w_s \supseteq v$ such that it contains the same words as $v \cup \{z\}$ and $\|w_s\| = 2p_{i+j}(p_k(n)) + 1$.
(Meaning: The witness pair (A_m, B_m) is not reducible to $(L(N_i), L(N_j))$ via F_k . It must be shown that z can be chosen as stated. Intuitively, this follows from $r_{s-1}(i, j, 0) \neq 0$.)

► **Definition 23** (Desired Oracle). Define $O := \bigcup_{s \in \mathbb{N}} w_s$ and $r := \bigcup_{s \in \mathbb{N}} r_s$.

It remains to show that Definition 23 is well-defined (i.e., that all steps of the oracle construction can be performed and makes real progress) and that O satisfies the desired properties. In Lemma 24, we show that O can be constructed as stated, in particular, we show that Case 3 of task $\tau_{i,0,0}$ and Case 1 of task $\tau_{i,j,k}$ are possible. The Propositions 29, 30, and 31 show that O satisfies all the desired properties.

► **Lemma 24.** For all $s \in \mathbb{N}$ it holds that w_s, r_s are well-defined, $w_s \subsetneq w_{s+1}$, $r_s \subsetneq r_{s+1}$ and that w_s is r_s -valid.

Proof. It is clear that w_0 and r_0 are well-defined and that w_0 is r_0 -valid. Let $s \in \mathbb{N}^+$ and $w_{s'}$ be $r_{s'}$ -valid for $s' < s$. The following three claims show that w_s is both well-defined and r_s -valid when defined by any given task. Let $i, j, k \in \mathbb{N}^+$.

▷ **Claim 25.** w_s is well-defined and r_s -valid when defined by task $\tau_{i,0,0}$.

Proof sketch. When the task is treated by the Cases 1 or 2, then the claim follows from the choice of w_s and Observation 20. For Case 3, the well-definedness follows from Case 2 failing and Observation 20. The r_s -validity follows from carefully choosing an r_{s-1} -valid oracle v' and from Observation 20.

▷ **Claim 26.** w_s is well-defined and r_s -valid when defined by task $\tau_{i,j,0}$.

Proof. If w_s is defined by Case 1 of task $\tau_{i,j,0}$ then w_s is explicitly chosen as an r_{s-1} -valid extension of w_{s-1} . Furthermore, no additional requirement must hold by the extension of r_{s-1} to r_s via $\{(i, j, 0) \mapsto 0\}$. So w_s is also r_s -valid.

Otherwise $w_s = w_{s-1}0$. By Observation 20, w_s remains r_{s-1} -valid. The extension of r_{s-1} to r_s via $\{(i, j, 0) \mapsto |w_s| + 2\}$ affects V4. Since $\|w_s\| < r_s(i, j, 0)$, w_s is not defined for words of any length in H_m . Hence, $V4(w_s, r_s)$ also remains satisfied. Consequently, w_s is r_s -valid also in this case. $\triangleleft$

▷ **Claim 27.** w_s is well-defined and r_s -valid when defined by task $\tau_{i,j,k}$.

Proof sketch. The main challenge is to show that choosing an appropriate $z \in 0\Sigma^{n-1}$ for some $n \in \mathbb{N}$ is possible. We prove the existence of such z by a proof by contradiction. If no choice of z satisfies neither Statement 1a nor Statement 1b, then $N_i(F_k(0^n))$ and $N_j(F_k(0^n))$ must accept for at least 2^{n-2} pairwise different choices of z respectively. Since every such accepting path queries at most polynomially many words, it is possible to choose $z_0 \in 00\Sigma^{n-2}$ and $z_1 \in 01\Sigma^{n-2}$ such that $N_i(F_k(0^n))$ does not query z_1 and $N_j(F_k(0^n))$ does not query z_0 . Hence, by adding z_0 and z_1 to the oracle, both $N_i(F_k(0^n))$ and $N_j(F_k(0^n))$ accept. From this we infer a contradiction on how the task $\tau_{i,j,0}$ was treated, as Case 1 would have already ruled out N_i and N_j to work disjoint. ◀

► **Lemma 28.** O is r -valid.

Proof. Follows from Lemma 24 and that any violation of V1 to V4 would also be a violation for some pair (w_s, r_s) with $s \in \mathbb{N}$. ◀

► **Proposition 29.** $O \in \text{SPARSE}$.

Proof. Follows by V1(O) which holds by Lemma 28. ◀

► **Proposition 30.** There are no recursive jump operators for TAUT relative to $O \cup A$.

Proof sketch. We assume that there is some recursive jump operator J_i and derive a contradiction by analyzing the treatment of task $\tau_{i,0,0}$. The Cases 1 and 2 achieve this contradiction almost by definition: the latter by showing that J_i is not recursive and the former by showing that J_i maps some proof system P_a for TAUT onto some transducer P_b which is no proof system for TAUT. Case 3 achieves a contradiction by showing that J_i maps some proof system for TAUT, namely $W_{i,m}$ for some m , onto some transducer P_b which is simulated by $W_{i,m}$. The simulation is possible, because $W_{i,m}$ uses Kleene's fixed point theorem to compute this mapping of J_i itself. Now, either $\text{ran}(P_b) \subseteq \text{TAUT}$ and $W_{i,m}$ can safely simulate P_b . Or $\text{ran}(P_b) \not\subseteq \text{TAUT}$, then we show that $\tau_{i,0,0}$ would have been treated by Case 1. The latter is enabled by the possibility to revoke the permission of $W_{i,m}$ to simulate P_b via words of the form $\langle 1^+, 1^+, 1^* \rangle$ inside the oracle. In total, all options to treat $\tau_{i,0,0}$ lead to a contradiction.

► **Proposition 31.** There are no $\leq_m^{\text{PP}}$ -complete disjoint NP-pairs relative to $O \cup A$.

Proof sketch. We assume that there is some $\leq_m^{\text{PP}}$ -complete pair $(L(N_i), L(N_j))$ and derive a contradiction by analyzing the treatment of the tasks $\tau_{i,j,0}$ and $\tau_{i,j,k}$ for $k \in \mathbb{N}^+$. When $\tau_{i,j,0}$ is treated by Case 1, then the oracle is chosen such that $L(N_i)$ and $L(N_j)$ are not disjoint. Otherwise, the tasks $\tau_{i,j,1}, \tau_{i,j,2}, \dots$ are treated. These tasks constructs the witness pair (A_m, B_m) such that it remains disjoint and such that it is not reducible to $(L(N_i), L(N_j))$ via $F_1, F_2, \dots$, thus ruling out the reducibility to $(L(N_i), L(N_j))$.

► **Theorem 32.** Let $A \subseteq 2\mathbb{N}$ be arbitrary. There is a sparse oracle $O \subseteq 2\mathbb{N} + 1$ relative to which $\text{DisjNP}^{O \cup A}$ has no $\leq_m^{\text{PP}}$ -complete pairs, $\text{TAUT}^{O \cup A}$ has no optimal proof systems, and $\text{TAUT}^{O \cup A}$ has no recursive jump operators relative to $O \cup A$.

Proof. Take oracle O from Definition 23. Then the properties for O and $O \cup A$ follow by Propositions 29, 30, 31, and the fact that the non-existence of $\leq_m^{\text{PP}}$ -complete pairs for DisjNP relativizably implies that TAUT has no optimal proof systems by Razborov [45]. ◀

► **Corollary 33.** *There is an oracle B relative to which PH^B is infinite, DisjNP^B has no $\leq_m^{\text{PP}}$ -complete pairs, TAUT^B has no optimal proof systems, and TAUT^B has no recursive jump operators relative to B .*

Proof. Let A be the oracle of Yao [50] relative to which the polynomial-time hierarchy is infinite. Without loss of generality, $A \subseteq 2\mathbb{N}$. Let $O \subseteq 2\mathbb{N} + 1$ be the sparse oracle from Theorem 32 such that the properties of Theorem 32 hold relative to $O \cup A$.

By results of Balcázar, Book, and Schöning [2] and Long and Selman [38], if the polynomial-time hierarchy is infinite, then the polynomial-time hierarchy is also infinite relative to any sparse oracle. Egidy and Glaßer [19, Cor. 3.18] show that this result allows us to combine above oracle $A \subseteq 2\mathbb{N}$ relative to which the polynomial-time hierarchy is infinite with above sparse oracle $O \subseteq 2\mathbb{N} + 1$ and the polynomial-time hierarchy remains infinite relative to $O \cup A$. Then $B := O \cup A$ is the desired oracle of this theorem. ◀

► **Corollary 34.** *Question Q1 can not be answered in the positive by relativizable means, even when assuming that the polynomial-time hierarchy is infinite.*

References

- 1 Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, USA, 1st edition, 2009.
- 2 Jose L. Balcázar, Ronald V. Book, and Uwe Schöning. The polynomial-time hierarchy and sparse oracles. *J. ACM*, 33(3):603–617, 1986. doi:10.1145/5925.5937.
- 3 O. Beyersdorff. Representable disjoint NP-pairs. In *Proceedings 24th International Conference on Foundations of Software Technology and Theoretical Computer Science*, volume 3328 of *Lecture Notes in Computer Science*, pages 122–134. Springer, 2004. doi:10.1007/978-3-540-30538-5_11.
- 4 O. Beyersdorff. Disjoint NP-pairs from propositional proof systems. In *Proceedings of Third International Conference on Theory and Applications of Models of Computation*, volume 3959 of *Lecture Notes in Computer Science*, pages 236–247. Springer, 2006. doi:10.18452/15520.
- 5 O. Beyersdorff. Classes of representable disjoint NP-pairs. *Theoretical Computer Science*, 377(1-3):93–109, 2007. doi:10.1016/j.tcs.2007.02.005.
- 6 O. Beyersdorff. The deduction theorem for strong propositional proof systems. *Theory of Computing Systems*, 47(1):162–178, 2010. doi:10.1007/s00224-008-9146-6.
- 7 Olaf Beyersdorff. On the existence of complete disjoint NP-pairs. In *11th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing*, pages 282–289, 2009. doi:10.1109/SYNASC.2009.9.
- 8 Olaf Beyersdorff, Johannes Köbler, and Jochen Messner. Nondeterministic functions and the existence of optimal proof systems. *Theoretical Computer Science*, 410(38):3839–3855, 2009. doi:10.1016/j.tcs.2009.05.021.
- 9 Olaf Beyersdorff and Zenon Sadowski. Do there exist complete sets for promise classes? *Mathematical Logic Quarterly*, 57(6):535–550, 2011. doi:10.1002/malq.201010021.
- 10 Sam R. Buss. *Bounded Arithmetic*. PhD thesis, Princeton University, 1985.
- 11 Yijia Chen and Jörg Flum. On p-optimal proof systems and logics for PTIME. In *Automata, Languages and Programming*, pages 321–332, 2010. doi:10.1007/978-3-642-14162-1_27.
- 12 Yijia Chen and Jörg Flum. On slice-wise monotone parameterized problems and optimal proof systems for TAUT. In *Computer Science Logic*, pages 200–214, 2010. doi:10.1007/978-3-642-15205-4_18.
- 13 Yijia Chen, Jörg Flum, and Moritz Müller. Hard instances of algorithms and proof systems. *ACM Trans. Comput. Theory*, 6(2), May 2014. doi:10.1145/2601336.
- 14 Stephen A. Cook and Robert A. Reckhow. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, 44:36–50, 1979. doi:10.2307/2273702.

- 15 David Dingel, Fabian Egidy, and Christian Glaßer. An oracle with no UP-complete sets, but $NP = PSPACE$. In *49th International Symposium on Mathematical Foundations of Computer Science (MFCS 2024)*, volume 306, pages 50:1–50:17, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.MFCS.2024.50.
- 16 Titus Dose. Further oracles separating conjectures about incompleteness in the finite domain. *Theoretical Computer Science*, 847:76–94, 2020. doi:10.1016/j.tcs.2020.09.040.
- 17 Titus Dose. An oracle separating conjectures about incompleteness in the finite domain. *Theoretical Computer Science*, 809:466–481, 2020. doi:10.1016/j.tcs.2020.01.003.
- 18 Titus Dose and Christian Glaßer. NP-completeness, proof systems, and disjoint NP-pairs. In C. Paul and M. Bläser, editors, *37th International Symposium on Theoretical Aspects of Computer Science, STACS 2020, March 10-13, 2020, Montpellier, France*, volume 154 of *LIPIcs*, pages 9:1–9:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPIcs.STACS.2020.9.
- 19 Fabian Egidy and Christian Glaßer. Optimal proof systems for complex sets are hard to find. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing, STOC '25*, pages 1329–1340, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3717823.3718182.
- 20 Anton Ehrmanntraut, Fabian Egidy, and Christian Glaßer. Oracle with $P = NP \cap coNP$, but no many-one completeness in UP, DisjNP, and DisjCoNP. In S. Szeider, R. Ganian, and A. Silva, editors, *47th International Symposium on Mathematical Foundations of Computer Science (MFCS 2022)*, volume 241 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 45:1–45:15, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.MFCS.2022.45.
- 21 C. Glaßer, A. L. Selman, and L. Zhang. Canonical disjoint NP-pairs of propositional proof systems. *Theoretical Computer Science*, 370:60–73, 2007. doi:10.1007/11549345_35.
- 22 C. Glaßer, A. L. Selman, and L. Zhang. The informational content of canonical disjoint NP-pairs. *International Journal of Foundations of Computer Science*, 20(3):501–522, 2009. doi:10.1007/978-3-540-73545-8_31.
- 23 Christian Glaßer, Alan L. Selman, Samik Sengupta, and Liyu Zhang. Disjoint NP-pairs. *SIAM Journal on Computing*, 33(6):1369–1416, 2004. doi:10.1137/S0097539703425848.
- 24 Christian Glaßer, Andrew Hughes, Alan L. Selman, and Nils Wisiol. *Disjoint NP-pairs and propositional proof systems*, pages 259–281. World Scientific, 2023. doi:10.1142/9789811278631_0010.
- 25 J. Grollmann and A. L. Selman. Complexity measures for public-key cryptosystems. *SIAM Journal on Computing*, 17(2):309–335, 1988. doi:10.1137/0217018.
- 26 Edward A. Hirsch. Optimal acceptors and optimal proof systems. In *Proceedings of the 7th Annual Conference on Theory and Applications of Models of Computation, TAMC'10*, pages 28–39, Berlin, Heidelberg, 2010. Springer-Verlag. doi:10.1007/978-3-642-13562-0_4.
- 27 Erfan Khaniki. New relations and separations of conjectures about incompleteness in the finite domain. *The Journal of Symbolic Logic*, 87(3):912–937, 2022. doi:10.1017/jsl.2021.99.
- 28 Erfan Khaniki. Jump operators, interactive proofs and proof complexity generators. In *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 573–593, 2024. doi:10.1109/FOCS61266.2024.00044.
- 29 Stephen Cole Kleene. On notation for ordinal numbers. *The Journal of Symbolic Logic*, 3(4):150–155, 1938. doi:10.2307/2267778.
- 30 Stephen Cole Kleene. *Introduction to Metamathematics*, volume 1 of *Bibliotheca Mathematica*. North-Holland Publishing Co., Amsterdam, 1952. doi:10.2307/2268620.
- 31 Johannes Köbler and Jochen Messner. Complete problems for promise classes by optimal proof systems for test sets. In *Proceedings of the 13th Annual IEEE Conference on Computational Complexity*, pages 132–140, 1998. doi:10.1109/CCC.1998.694599.

- 32 J. Krajíček. *Bounded Arithmetic, Propositional Logic and Complexity Theory*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1995. doi:10.1017/CB09780511529948.
- 33 Jan Krajíček. Diagonalization in proof complexity. *Fundamenta Mathematicae*, 182:387–397, 2004. doi:10.4064/fm182-2-7.
- 34 Jan Krajíček. Implicit proofs. *Journal of Symbolic Logic*, 69(2):387–397, 2004. doi:10.2178/jsl/1082418532.
- 35 Jan Krajíček. *Proof Complexity*. Cambridge University Press, 2019. doi:10.1017/9781108242066.
- 36 Jan Krajíček and Pavel Pudlák. Propositional proof systems, the consistency of first order theories and the complexity of computations. *Journal of Symbolic Logic*, 54:1063–1079, 1989. doi:10.2307/2274765.
- 37 Johannes Köbler, Jochen Messner, and Jacobo Torán. Optimal proof systems imply complete sets for promise classes. *Information and Computation*, 184(1):71–92, 2003. doi:10.1016/S0890-5401(03)00058-0.
- 38 Timothy J. Long and Alan L. Selman. Relativizing complexity classes with sparse oracles. *Journal of the ACM*, 33(3):618–627, 1986. doi:10.1145/5925.5938.
- 39 Jochen Messner. On optimal algorithms and optimal proof systems. In *Symposium on Theoretical Aspects of Computer Science*, pages 541–550, Berlin, Heidelberg, 1999. Springer-Verlag. doi:10.1007/3-540-49116-3_51.
- 40 Jochen Messner. *On the Simulation Order of Proof Systems*. PhD thesis, Universität Ulm, 2000.
- 41 Ján Pich and Rahul Santhanam. Learning algorithms versus automatability of frege systems. In *49th International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 229, pages 101:1–101:20, 2022. doi:10.4230/LIPIcs.ICALP.2022.101.
- 42 P. Pudlák. On reducibility and symmetry of disjoint NP pairs. *Theoretical Computer Science*, 295:323–339, 2003. doi:10.1016/S0304-3975(02)00411-5.
- 43 Pavel Pudlák. Incompleteness in the finite domain. *The Bulletin of Symbolic Logic*, 23(4):405–441, 2017. doi:10.1017/bsl.2017.32.
- 44 P. Pudlák. The lengths of proofs. In S. R. Buss, editor, *Handbook of Proof Theory*, pages 547–637. Elsevier, Amsterdam, 1998.
- 45 Alexander Razborov. On provably disjoint NP-pairs. Technical Report TR94-006, Electronic Colloquium on Computational Complexity, 1994. URL: <https://eccc.weizmann.ac.il/report/1994/006/>.
- 46 Zenon Sadowski. On an optimal quantified propositional proof system and a complete language for $NP \cap coNP$. In *Fundamentals of Computation Theory*, volume 1279, pages 423–428. Springer, 1997. doi:10.1007/BFB0036203.
- 47 Zenon Sadowski. On an optimal deterministic algorithm for sat. In *Computer Science Logic*, pages 179–187, Berlin, Heidelberg, 1999. Springer-Verlag. doi:10.1007/10703163_13.
- 48 A. L. Selman. Promise problems complete for complexity classes. *Information and Computation*, 78:87–98, 1988. doi:10.1016/0890-5401(88)90030-2.
- 49 Robert I. Soare. *Recursively Enumerable Sets and Degrees*. Perspectives in Mathematical Logic. Springer-Verlag, Berlin, 1987. doi:10.1007/978-3-662-02460-7.
- 50 Andrew Yao. Separating the polynomial-time hierarchy by oracles. In *Proceedings of the 26th Annual Symposium on Foundations of Computer Science*, SFCS '85, pages 1–10, USA, 1985. IEEE Computer Society. doi:10.1109/SFCS.1985.49.